Chronological Networks as a Library Primitive: ChronoLog over an Interval-Native Date/Time Stack

Kip Cole ✉
Independent

Abstract

Levy, Geeraerts, Pluquet, Piasetzky and Fantalkin (2020) formalised archaeological chronology as a *chronological network* — a set of time-periods related by synchronisms, asynchronisms, and duration bounds — and showed that consistency checking and the computation of tightest possible date ranges reduce to a Simple Temporal Problem (STP), solvable in polynomial time. Their tool, ChronoLog, invents an interval/relation vocabulary from scratch. We observe that this vocabulary is exactly the one a general, interval-native date/time library already provides: half-open intervals, Allen's thirteen relations, partial and reduced-precision dates, calendar-aware durations, and uncertainty qualifiers. We implement the ChronoLog model as a small layer (`Tempo.Network`) over the Tempo Elixir library, mapping each ChronoLog relation to its Allen counterpart plus metric (delay) extensions, normalising a network to an STP, and solving consistency and tightening by Floyd–Warshall. The layer reproduces the paper's published bounds exactly and inherits, for free, ISO 8601 / EDTF interchange, multiple calendars, BCE arithmetic, and §8 uncertainty qualifiers. The contribution is less a new algorithm than an argument: chronological-network modelling is a natural *library primitive* when the library already treats time as intervals.

2012 ACM Subject Classification Applied computing → Arts and humanities; Theory of computation → Modal and temporal logics; Information systems → Temporal data

Keywords and phrases Chronological networks, Allen's interval algebra, simple temporal problems, archaeological dating, ISO 8601, EDTF, interval-based time

Digital Object Identifier 10.4230/OASIcs...

Related Version Source repository: `https://github.com/kipcole9/tempo`

1 Introduction

Archaeological and historical chronology is rarely a list of fixed dates. It is a web of relationships: a king reigns before another, a stratum is built during a reign, a ceramic style overlaps its successor, an inscription supplies a *terminus post quem*. Levy et al. [6] call such a web a *chronological network* and give it a formal model with three object types — time-periods, sequences, and chronological relations — and two operations: *consistency checking* (does any assignment of dates satisfy every constraint?) and *tightening* (what is the narrowest start, end, and duration each period can take?). Their key technical result is that the model is a Simple Temporal Problem [2]: every relation, duration, and absolute date reduces to atomic constraints of the form $b_1 - b_2 \leq k$ over boundary variables, so consistency is the absence of a negative cycle and tightening is all-pairs shortest paths. This is polynomial-time, in contrast to exhaustive-search tools such as Groundhog [3].

ChronoLog [7] implements this model as a bespoke application, and in doing so re-creates a small interval algebra: a period has a start, an end, and a duration, each possibly unknown, bounded, or ranged; relations express containment, overlap, synchronised boundaries, and metric delays. Our observation is that this is precisely the vocabulary that an *interval-native* general-purpose date/time library already provides. We demonstrate this with Tempo, an Elixir library in which every temporal value is a bounded half-open interval at a declared

resolution, comparison is Allen's thirteen relations [1], and partial ISO 8601 / EDTF values (2022, 2022-06, ∼720) are first-class.

We contribute `Tempo.Network`, a layer that (i) names the ChronoLog relation vocabulary and maps it to Allen's algebra plus metric extensions (§4); (ii) normalises a network to an STP against a single origin boundary (§5); and (iii) solves consistency and tightening by Floyd–Warshall, with shortest-path *traces* that explain each derived bound (§5). The layer reproduces the paper's ChronoLand and 26th-dynasty results exactly (§6) and inherits ISO 8601/EDTF interchange, calendars, and qualifiers from the host library (§7).

2 The ChronoLog model

A *time-period* is a continuous interval — a reign, an era, the life of a stratum — characterised by a start date, an end date, and a duration, each of which may be unknown, known, lower-bounded, upper-bounded, or known within a range $[lo, hi]$. Dates and durations are modelled *independently*: a period is at most six numbers (earliest/latest start, earliest/latest end, minimum/maximum duration), reconciled by the relation $duration = end - start$.

A *sequence* is a list of consecutive time-periods with no gaps: $end(p_i) = start(p_{i+1})$. A gap is modelled by inserting an explicit gap period.

A *chronological relation* constrains two periods. The vocabulary (the paper's Tables 1–5) covers contemporaneity ($A \sim B$: non-empty overlap), inclusion ($A \subseteq B$, $A \supseteq B$), overlap ($A \leq B$, $A \geq B$), start/end-period synchronisms ("A starts during B", "A includes the start of B", and the end-side duals), synchronised boundaries (synchronous start/end, equality, immediate precedence/following), asynchronisms ($A \ll B$, $A \gg B$), ordered boundaries (strict orderings of individual start/end dates), and *delay synchronisms* — metric constraints stating that one boundary is an exact, minimum, or maximum number of years before or after another.

2.0.0.1 The STP reduction.

Every relation, duration, sequence link, and absolute date normalises to a conjunction of atomic constraints of one shape, $b_1 - b_2 \leq k$, where b_1, b_2 are boundary variables and k is an integer. Equalities become two inequalities; a strict $<$ on an integer time-scale becomes $\leq k - 1$; absolute dates are constraints against a single origin boundary z_0. The constraints form a directed weighted graph with one node per boundary plus z_0. The network is *consistent* iff the graph has no negative cycle, and the tightest bound on $b_1 - b_2$ is the shortest-path weight $b_1 \to b_2$; tightening is therefore all-pairs shortest paths (Floyd–Warshall) [2, 4].

3 Tempo's interval model

Tempo represents every temporal value as a bounded, half-open interval $[from, to)$ at a declared resolution. A partial ISO 8601 value is an interval of the appropriate width: `2026` is $[2026\text{-}01\text{-}01, 2027\text{-}01\text{-}01)$, `2026-06` spans one month, `2026-06-15` one day. The half-open convention makes adjacent spans concatenate without gap or overlap, and makes comparison total under Allen's thirteen relations, exposed as predicates (`before?`, `overlaps?`, `within?`, `adjacent?`). Durations are calendar-aware (`P20Y`, `P2Y6M`), and ISO 8601-2 / EDTF uncertainty qualifiers (? uncertain, ∼ approximate, % both) are parsed and carried per component.

These are exactly the primitives ChronoLog needs. A time-period's boundaries are interval endpoints; a sequence is the half-open concatenation $[a, b)$ then $[b, c)$; the relations

Table 1 Representative ChronoLog relations, their Allen counterpart, and their atomic constraints; the boundary and delay rows stand for a parameterised family. s, e abbreviate start, end.

ChronoLog relation	Allen	Atomic constraints
$A \ll B$ (before)	precedes	$e(A) - s(B) \leq -1$
$A \sqsupset B$ (immed. prec.)	meets	$e(A) = s(B)$
$A \subseteq B$ (included in)	during	$s(B) \leq s(A),\ e(A) \leq e(B)$
$A \supseteq B$ (includes)	contains	$s(A) \leq s(B),\ e(B) \leq e(A)$
$A \leq B$ (overlaps)	overlaps	$s(A) \leq s(B) \leq e(A) \leq e(B)$
$A \sim B$ (contemporary)	—	$s(B) \leq e(A),\ s(A) \leq e(B)$
A begins B (starts)	starts	$s(A) = s(B),\ e(A) \leq e(B)$
A starts during B	—	$s(B) \leq s(A) \leq e(B)$
A ends during B	—	$s(B) \leq e(A) \leq e(B)$
$A \top B$ (synchronous start)	—	$s(A) = s(B)$
boundary: $e(A) \leq s(B)$	—	$e(A) - s(B) \leq 0$
delay: $s(A)$ at least X before $s(B)$	—	$s(A) - s(B) \leq -X$

are Allen relations plus metric offsets; absolute and floating chronologies are anchored and unanchored values; uncertainty is the qualifier vocabulary. The one missing piece is the constraint solver, which is small and well-understood. `Tempo.Network` supplies it.

4 Mapping the relation vocabulary

Each ChronoLog relation is stored as data and translated, by `to_atomic/1`, to its atomic constraints; Table 1 gives the correspondence to Allen's algebra and the atomic form. Where a single Allen relation exists the mapping is one-to-one (and invertible, so a solved network can be *queried* with Allen predicates and an Allen relation between two solved periods can be *named*). The period synchronisms ("starts during" and kin) are looser than any single Allen relation — they are disjunctions — and the metric delays carry an integer Allen cannot express.

The coverage is *complete* for ChronoLog's boundary vocabulary, not a selection from it. Every synchronism reduces to one of three mechanisms. (i) The qualitative relations include all thirteen of Allen's — the four precise endpoint-sharing relations (*starts*, *started-by*, *finishes*, *finished-by*) alongside containment, overlap, immediate precedence, and the looser period synchronisms. (ii) A single parameterised *boundary* relation compares one endpoint of A with one endpoint of B under any of $\{<, \leq, =, \geq, >\}$; this one relation form expresses ChronoLog's entire family of ordered single-boundary synchronisms ("starts/ends strictly or loosely before/after/at the start/end of", sixteen in all). (iii) The metric *delay* relation states an exact, minimum, or maximum offset between two endpoints. ChronoLog's *strict composite* synchronisms (e.g. strictly-included-in) are a non-strict relation conjoined with one strict boundary relation; the only family left outside the layer is its tolerance and gazetteer-hierarchy relations (*within*, *child-of*, *parent-of*), which are tied to ChronoLog's period-database features rather than to interval geometry. We validate the mapping against ChronoLog's own published models (§6).

5 Normalisation and solver

`Tempo.Network.Normalize` reduces a network to $\{nodes, edges, unit\}$. Each period contributes `start` and `end` nodes; a shared origin `:origin` (z_0) anchors absolute dates. The network's *finest unit* — the finest resolution among its dates — fixes the integer axis: a date becomes its count of that unit relative to z_0, a duration its count of that unit. Conversion is exact for the uniform-unit (typically year-grained) chronologies that are the discipline's norm; a duration expressed across units uses nominal calendar lengths. Each edge is tagged with the constraint that produced it, for tracing.

`Tempo.Network.Solver` runs Floyd–Warshall once, in $O(n^3)$ on the boundary count — interactive for the hundreds of periods these chronologies contain. `consistent?/1` reports the absence of a negative cycle; `tighten/1` reads the tightest start, end, and duration of each period off the shortest-path matrix and maps them back to Tempo values. `trace/3` reconstructs the shortest path underlying a chosen bound and renders it as prose — the paper's notion of an explanatory trace, where every step names the input it rests on.

6 Worked examples

6.0.0.1 ChronoLand.

The paper's toy kingdom: kings `K1` then `K2` reign in succession, both between 1200 and 1300 CE; `K1` reigns at most 10 years, `K2` at least 35. Strata `S1` (built by `K1`) and `S2` (destroyed under `K2`) follow one another, each lasting 20–100 years.

```
network =
  Network.new()
  |> Network.add_period(:k1, start: {:not_before, ~o"1200Y"}, duration: {:at_most, ~o"P10Y"})
  |> Network.add_period(:k2, end: {:not_after, ~o"1300Y"}, duration: {:at_least, ~o"P35Y"})
  |> Network.add_period(:s1, duration: {~o"P20Y", ~o"P100Y"})
  |> Network.add_period(:s2, duration: {~o"P20Y", ~o"P100Y"})
  |> Network.add_sequence([:k1, :k2])
  |> Network.add_sequence([:s1, :s2])
  |> Network.add_relation(:starts_during, :s1, :k1)
  |> Network.add_relation(:ends_during, :s2, :k2)
```

Tightening reproduces the paper's Fig. 6b exactly: `K2` tightens to start $\in [1200, 1265]$, end $\in [1240, 1300]$, duration $\in [35, 100]$; the strata, given no dates as input, acquire them, and each stratum's duration tightens from $[20, 100]$ to $[20, 80]$ because the dynasty is too short to hold a longer one. The trace for the earliest end of `K2` reproduces the paper's Fig. 6c reasoning: `K1` starts no earlier than 1200; `S1` starts during `K1`; `S1` lasts at least 20 years, so ends after 1220; `S1` meets `S2`; `S2` lasts at least 20 years, so ends after 1240; `S2` ends during `K2`, so `K2` ends after 1240. Capping `K2` at 25 rather than 35 years makes the network inconsistent (Fig. 7), detected as a negative cycle.

6.0.0.2 The 26th dynasty.

Six reigns in succession (after Kitchen 2000). Given only Psammetichus I's accession (664 BCE) and each reign's length, tightening derives every reign's absolute dates — exercising BCE (negative) years and a six-period sequence. Amasis II's accession is recovered as 570 BCE, forced by the anchor and the five preceding reigns, with no date stated for it directly.

6.0.0.3 A native-format corpus.

Beyond the two worked examples, we exercise the layer against a corpus of published ChronoLog models in the tool's own save format. ChronoLog's `.clog` files are JSON; we decode their periods, sequences, events, and synchronisms directly into networks and re-solve them. The corpus spans the 26th-dynasty model above, a New-Kingdom Dynasty 18 model, an Aegean Late-Helladic to Proto-Geometric ceramic sequence, an Iron Age Levant model, and a Mediterranean Late Bronze Age study. Every model decodes and is consistent. The two that carry historical anchors *inside* the network — the 26th dynasty (anchored by the Persian conquest of 525 BCE together with a set of epigraphic Apis-bull delay synchronisms) and the Mediterranean study (dated events and an Amarna delay) — tighten to absolute calendar dates, the former recovering its full reign chronology exactly. The remaining three are radiocarbon-driven and hence purely *relative*: their absolute evidence is radiocarbon, which lies outside the network (§7), so the layer confirms their consistency and relative ordering but leaves them unanchored in absolute time until a historical date is supplied. That a single decoder both reproduces an anchored chronology to the year and correctly reports a relative one as floating is itself evidence that ChronoLog's serialised models and the interval layer express the same object.

7 What the library adds, and its limits

Because boundaries are ordinary Tempo values, a network is interoperable by construction: each period round-trips through ISO 8601 / EDTF for exchange with other tooling, BCE and proleptic dates are handled uniformly, and non-Gregorian calendars are available through the underlying calendrical stack. Uncertainty qualifiers are carried as metadata and, by design, do *not* silently widen a bound: a vague "circa" never performs arithmetic unless the modeller restates it as a range. The solver shares interval-arithmetic primitives with the library's planned set-operations milestone (union, intersection, coalescing of interval sets).

7.0.0.1 Scope: two layers.

A chronology has a *relative/metric* layer — the order, duration, and synchronism constraints that fix periods relative to one another and to historical anchors — and an *absolute/scientific* layer, in which radiocarbon determinations are calibrated against a reference curve (IntCal) and combined by Bayesian modelling (OxCal) into probabilistic calendar ranges. `Tempo.Network` is the first layer in full: it derives *deterministic* bounds from order, duration, and anchors. It is deliberately not the second — it neither calibrates radiocarbon nor computes probability distributions, and ChronoLog's per-period radiocarbon samples (its `C14...ForExport` fields, destined for OxCal) are carried through decoding but excluded from the constraint graph. Uncertainty here is bounds, not probability; this is why a radiocarbon-only model (§6) is consistent and internally ordered yet floating in absolute time. The two layers compose — the relative network is the natural prior structure for the Bayesian one — but the statistical layer belongs to a tool such as OxCal, not to an interval library.

7.0.0.2 Limitations.

Following the source model, relations compose by *conjunction*; a genuine *or*-relation ("A ends before B *or* A starts after B") is outside the STP and is not supported. Cross-unit duration conversion (e.g. a year-grained duration on a day axis) uses nominal calendar lengths and

is therefore approximate, though the uniform-unit chronologies that dominate practice are exact.

8 Related work

Allen's interval algebra [1] underlies the relation vocabulary; Holst [5] applied temporal logic to archaeology. The direct antecedent is ChronoLog [6, 7]; the STP machinery is that of Dechter, Meiri and Pearl [2] via Floyd [4]. Falk's Groundhog [3] addresses the same problem by exhaustive search. Tempo's contribution is orthogonal to all of these: it shows that, given an interval-native host library, the network model and its solver are a compact, reusable primitive rather than a standalone application.

9 Conclusion

Chronological-network modelling does not require a bespoke tool. When the host library already represents time as half-open intervals with Allen's algebra, partial dates, and calendar-aware durations, the ChronoLog model is a thin layer: a relation-to-atomic translation, an STP normalisation, and a Floyd–Warshall solver. The result reproduces the reference results exactly while inheriting ISO 8601/EDTF interchange, calendars, and uncertainty qualifiers — and makes consistency checking and tightening available wherever the library is.

References

1 James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11):832–843, 1983. doi:10.1145/182.358434.

2 Rina Dechter, Itay Meiri, and Judea Pearl. Temporal constraint networks. *Artificial Intelligence*, 49(1–3):61–95, 1991. doi:10.1016/0004-3702(91)90006-6.

3 Dan Falk. Groundhog: A chronological modelling tool. `http://www.groundhogchronology.com/`, 2020.

4 Robert W. Floyd. Algorithm 97: Shortest path. *Communications of the ACM*, 5(6):345, 1962. doi:10.1145/367766.368168.

5 Mads K. Holst. Complicated relations and blind dating: Formal analysis of relative chronological structures. In *Tools for Constructing Chronologies*, pages 129–147. Springer, 2004.

6 Eythan Levy, Gilles Geeraerts, Frédéric Pluquet, Eli Piasetzky, and Alexander Fantalkin. Chronological networks in archaeology: A formalised scheme. *Journal of Archaeological Science*, 124:105225, 2020. doi:10.1016/j.jas.2020.105225.

7 ChronoLog. `http://chrono.ulb.be`.